P3

Reconstruir lógica y datos con Binary Ninja

Parte I · RA3 · 4,5 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Guion de práctica para el alumnado
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Índice

1. Mapa de trabajo **2**

2. Estado y material **2**

3. Trabajo en sesión (270 minutos) **2**

4. Flujo de herramienta **2**

5. Entregable **2**

6. Evaluación **3**

7. Teoría necesaria para realizar la práctica **3**

8. Software requerido **3**

9. Preparación de la base de análisis **3**

10. Guía detallada: calentamiento disponible **4**
10.1. 1. Triaje y referencias . . . 4
10.2. 2. Reconstruir entrada y control . . . 4
10.3. 3. Preparar cinco casos para P4 . . . 4

11. Guía detallada: RE-01 cuando esté disponible **4**
11.1. 1. Fijar preguntas . . . 4
11.2. 2. Seguir datos . . . 4
11.3. 3. Recuperar una estructura . . . 5
11.4. 4. Especificar validaciones y efectos . . . 5
11.5. 5. Preparar pruebas discriminantes . . . 5

12. Uso de la herramienta **5**

13. Controles de calidad y entrega **5**

14. Actualización RE-01 1.0 — 28 de septiembre de 2026 **5**
14.1. Material definitivo y recepción . . . 6
14.2. Reconstrucción y predicciones del ejercicio central . . . 6

Estado actual: RE-01 1.0 disponible. La actualización del 28 de septiembre de 2026, al final del documento, contiene el material definitivo y sustituye las menciones a RE-01 pendiente de producción. Las fichas anteriores se conservan como antecedente.

1 Mapa de trabajo



Figura 1: Secuencia de trabajo y puntos de control de la práctica.

El diagrama resume el orden recomendado. Cada transición debe quedar respaldada por una anotación, salida o fichero verificable en la entrega.

4,5 h · RA3 · 15 % de la nota de Parte I. Guion completo en LaTeX. Rúbrica única en criterios de evaluación.

2 Estado y material

Calentamiento disponible: `ejemplos-x86-strip-opt`, fuente publicado. Ejercicio central: RE-01 del caso integrador, **pendiente de producción**. Se debe suministrar antes de impartir la práctica completa. No se sustituye el objetivo central por cuatro horas de búsqueda de una cadena visible con `strings`.

3 Trabajo en sesión (270 minutos)

1. Calentamiento con el crackme y Binary Ninja (35 min): formato, cadenas, referencias, condición de longitud, bucle y salidas. La clave no se considera secreta.
2. Recepción de RE-01, triaje y preguntas (20 min).
3. Localizar entrada/configuración y funciones relevantes (50 min).
4. Recuperar tipos, una estructura y validaciones; anotar ensamblador y HLIL (70 min).
5. Escribir especificación de entradas, efectos y errores; cinco casos de prueba para P4, incluyendo vacío/límite/fuera de rango según contrato (55 min).
6. Revisión cruzada y exportación (40 min).

4 Flujo de herramienta

Abrir el binario y comprobar plataforma; esperar análisis; navegar cadenas y referencias hasta la lógica relevante. Renombrar funciones/variables con justificación, definir tipos y comentarios. Comparar una condición HLIL con ensamblador y consultar MLIL cuando aclare tamaño/signo. No se exige API ni automatización sin interfaz. Guardar `.bndb` y exportar extractos legibles con direcciones y hash de muestra.

5 Entregable

Especificación reconstruida, tabla de campos/tipos, grafo o fragmentos anotados, cinco predicciones y al menos dos incertidumbres que se contrastarán en P4. La deducción es estática; ejecutar posteriormente para contrastar se registra como otra fase, no invalida por sí mismo todo el trabajo. Se admite Ghidra como alternativa si se aportan evidencias equivalentes. No se califica solo la clave obtenida.

6 Evaluación

Se aplica la rúbrica común: corrección técnica 40 %, evidencia reproducible 25 %, interpretación y límites 25 %, claridad 10 %. Todos los criterios se aplican al objetivo concreto de esta práctica; las ampliaciones no son necesarias para la máxima nota.

7 Teoría necesaria para realizar la práctica

El análisis estático reconstruye una especificación a partir del fichero sin ejecutarlo. Se combinan cadenas, referencias cruzadas, grafo de llamadas, control, accesos a memoria e imports. El HLIL facilita la lectura; el ensamblador confirma tamaños, comparaciones y efectos que una descompilación puede simplificar.

Los nombres y tipos asignados son hipótesis. Un tipo gana apoyo si explica anchos, offsets, usos y llamadas. Una estructura se justifica por accesos repetidos desde una misma base. Las condiciones de éxito se reconstruyen siguiendo ambos caminos hasta retornos o efectos. Una cadena visible solo adquiere significado al estudiar sus referencias. La optimización puede integrar funciones o propagar constantes.

Componente de la especificación	Contenido esperado
Entrada	formato, longitud, rango y codificación
Estado	configuración, campos y ficheros consultados
Decisión	condiciones observadas y orden relevante
Salida	retorno, mensaje, fichero o llamada
Incertidumbre	interpretación que deberá contrastar P4

8 Software requerido

Obligatorio: Binary Ninja Desktop con licencia del curso, o Ghidra validado; `file`, `sha256sum`, `strings`, `readelf` y `objdump`.

```
bash PreparaciónEntorno/entorno/provision.sh --check
make -C muestras/build x86
mkdir -p resultados/P3
sha256sum muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P3/calentamiento.sha256
```

RE-01 solo se usa cuando el docente entregue fichero y manifiesto definitivos. El calentamiento es ejecutable ahora, pero no sustituye el ejercicio central pendiente.

9 Preparación de la base de análisis

1. Cree un directorio nuevo y una copia de trabajo.
2. Verifique el SHA-256 frente al manifiesto.
3. Confirme formato, arquitectura, endianness y base de análisis.
4. Espere el análisis automático y registre versión de herramienta.
5. Guarde una base inicial y otra final.

No ejecute el binario durante P3. La fuente pública del calentamiento se usa después como control; primero se documenta la deducción desde el binario.

10 Guía detallada: calentamiento disponible

10.1 1. Triaje y referencias

```
file muestras/build/salida/ejemplos-x86-strip-opt
readelf -h -l -S muestras/build/salida/ejemplos-x86-strip-opt
strings -a -t x muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P3/cadenas.txt
```

Localice mensajes de uso, éxito y error. Siga sus referencias a la lógica que decide qué mensaje imprimir. Una cadena que parece credencial no se acepta como solución hasta relacionarla con comparaciones o llamadas.

10.2 2. Reconstruir entrada y control

Parta de `main`: identifique `argc`, `argv` y la condición que exige argumento. Siga el puntero hasta la validación. Anote cómo se obtiene la longitud, el rechazo de longitudes distintas y la comparación byte a byte. Divida en bloques: precondición, bucle, diferencia, éxito y retorno.

Compruebe en ensamblador el registro del puntero, la anchura comparada, el límite del bucle y los retornos. Explique longitud de cadena frente a almacenamiento con NUL. Si la validación está integrada, documente el rango dentro de `main`.

10.3 3. Preparar cinco casos para P4

ID	Clase	Predicción	Rama o efecto que comprobará
T1	vacío	completar	precondición
T2	longitud corta/larga	completar	rechazo por longitud
T3	longitud correcta, primer byte distinto	completar	salida temprana
T4	prefijo correcto, byte tardío distinto	completar	iteración
T5	valor reconstruido	completar	éxito

No ejecute aún los casos. Seleccione además una condición HLIL y aporte dirección, instrucciones, flags/salto y pseudocódigo equivalente.

11 Guía detallada: RE-01 cuando esté disponible

11.1 1. Fijar preguntas

Identifique RE-01 por manifiesto. Determine qué configuración acepta, cómo valida longitud/rango, qué documento consulta, qué transformación realiza, qué salida produce y qué errores diferencia.

11.2 2. Seguir datos

Empiece en `main`, imports de ficheros/parseo y cadenas de error. Construya un grafo reducido de tres a cinco funciones:

Función/dirección	Entradas	Estado leído	Efectos/retorno	Apoyo
completar	completar	completar	completar	observación/inferencia

11.3 3. Recuperar una estructura

Agrupe accesos `base+offset`, determine ancho y uso y proponga la estructura mínima. Separe offset observado, tipo compatible, nombre propuesto y evidencia. No rellene huecos con campos inventados.

11.4 4. Especificar validaciones y efectos

Redacte precondiciones y postcondiciones. Para cada error indique localizador y si evita efectos posteriores. Determine el modo de apertura solo si llamadas y flags lo sostienen.

11.5 5. Preparar pruebas discriminantes

Incluya nominal válido, vacío, límites y fuera de rango según el contrato real. Cada prueba cambia una condición y predice retorno, mensaje y efectos para P4.

12 Uso de la herramienta

- Cadenas y referencias: entrada al contexto, no conclusión.
- Grafo: orientación; confirme llamadas indirectas e integración.
- HLIL: estructura legible; contraste signo y tamaño con MLIL/ensamblador.
- Tipos: aplíquelos gradualmente y revise todas sus referencias.
- Comentarios: registre evidencia y duda con localizador.
- Base `.bndb`: guárdela junto al hash del binario correspondiente.

Ghidra es válido con direcciones, referencias, tipos y fragmentos equivalentes.

13 Controles de calidad y entrega

Incluya identificación, mapa de funciones, especificación, estructura, condiciones, fragmentos anotados, cinco predicciones e incertidumbres. Antes de cerrar:

- cada nombre semántico tiene indicios;
- cada campo tiene offset y anchura;
- cada condición decisiva se comprobó en ensamblador;
- capacidad y ejecución histórica permanecen separadas;
- binario y base tienen relación inequívoca;
- no se rellenan datos de RE-01 mientras siga pendiente.

14 Actualización RE-01 1.0 — 28 de septiembre de 2026

RE-01 ya está producido. Esta sección actualiza las menciones anteriores a material pendiente, conservadas como antecedente. El calentamiento y su tabla T1–T5 se mantienen; las pruebas RE-01.T1–T5 siguientes pertenecen al ejercicio central.

14.1 Material definitivo y recepción

El docente entrega RE-01-alumnado.tar.gz. Incluye ELF stripped x86-64 PIE, siete entradas sintéticas, instrucciones, manifiesto y SHA256SUMS. Necesita Linux x86-64 con glibc compatible (requisito máximo GLIBC_2.34), Binary Ninja o Ghidra, file, readelf, objdump y sha256sum. No se necesita compilar. El docente conserva fuente y símbolos, que no forman parte de este archivo.

Desde un directorio nuevo en la VM, después de extraer el paquete:

```
cd alumnado
sha256sum -c SHA256SUMS
file RE-01
readelf -h -lW -SW RE-01 > inventario-elf.txt
```

El manifiesto declara 14.472 bytes. Compare el hash completo del fichero con MANIFIESTO.json antes de analizarlo; los localizadores de otros builds no sirven. Las entradas usan ASCII y LF. Las rutas de los tres argumentos son configuración, documento y resultado nuevo. El directorio de resultados debe existir.

14.2 Reconstrucción y predicciones del ejercicio central

1. Importe RE-01 como ELF x86-64. Anote la base elegida por la herramienta y distinga VA ELF, offset de fichero y futura dirección cargada.
2. Parta de cadenas de formato e imports para localizar entradas, validaciones y escritura. Dibuje el grafo de las cinco funciones relevantes; no cuente libc ni el código de arranque como funciones del caso.
3. Proponga estructuras con offsets, anchuras y tamaños. Justifique cada campo por lecturas y escrituras; contraste el signo de la comparación decisiva en ensamblador.
4. Reconstruya gramática, orden de campos, límites, transformación y cálculo de la suma. Explique qué cambia y qué conserva de las entradas.
5. Distinga retorno de proceso, stdout, stderr y fichero de salida. Determine el comportamiento de una ruta ya existente sin ejecutar todavía el programa.
6. Complete esta tabla con predicciones y VA ELF de las condiciones relevantes.

ID	Configuración en entradas/	Documento en entradas/	Predicción y condición
RE-01.T1	valida.cfg	documento.txt	completar
RE-01.T2	valida.cfg	vacio.txt	completar
RE-01.T3	limite.cfg	limite.txt	completar
RE-01.T4	limite.cfg	exceso.txt	completar
RE-01.T5	opcion-invalida.cfg	documento.txt	completar

Entregue la especificación provisional y estas cinco predicciones antes de P4. La especificación debe permitir implementar una utilidad equivalente para las entradas admitidas e identificar qué comportamiento aún necesita contraste.

Una incertidumbre útil es concreta y añade la operación que podría resolverla.